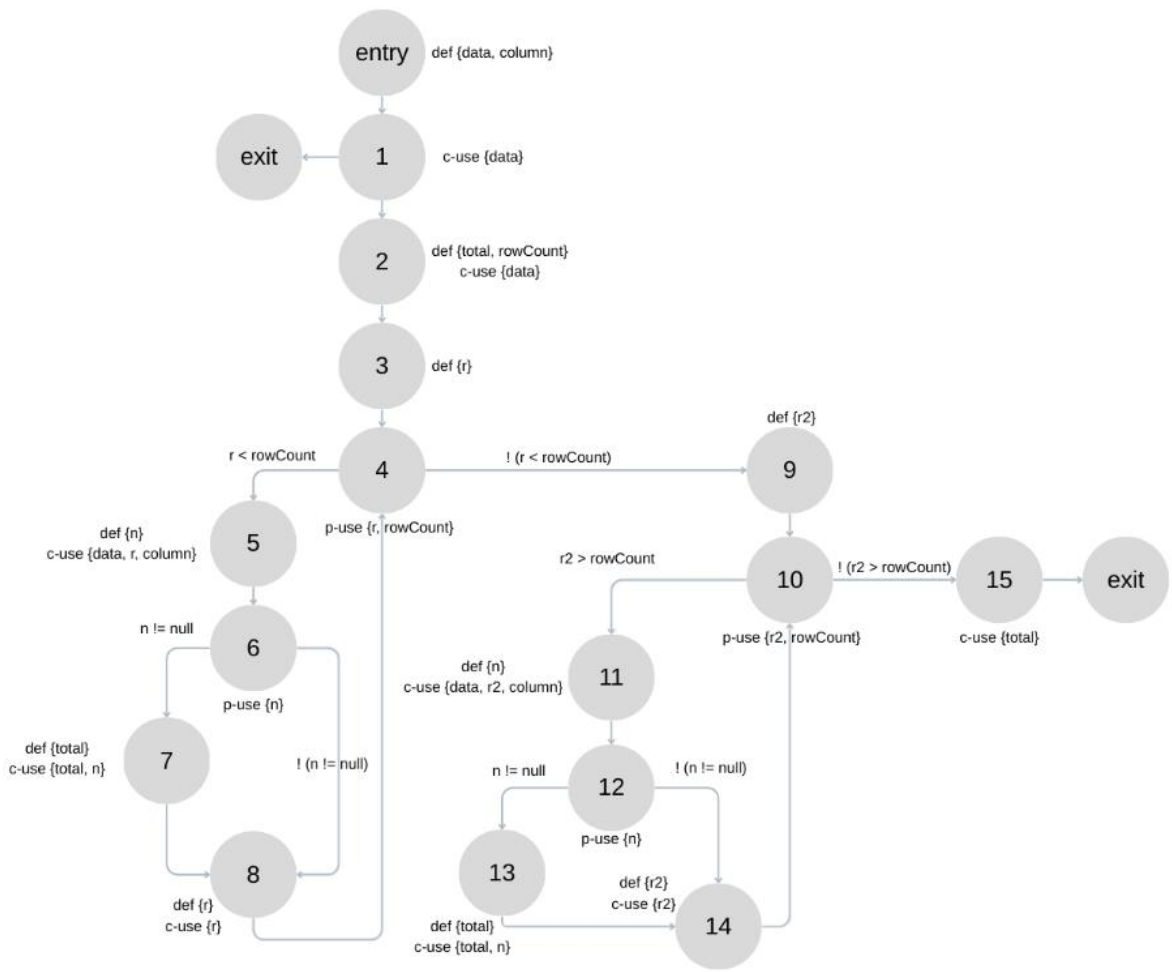
Manual Measurement of Data Flow Coverage

calculateColumnTotal(Values2D data, int column) method

```
public static double calculateColumnTotal(Values2D data, int column) {
    ParamChecks.nullNotPermitted(data, "data");
    double total = 0.0;
    int rowCount = data.getRowCount();
    for (int r = 0; r < rowCount; r++) {
        Number n = data.getValue(r, column);
        if (n != null) {
            total += n.doubleValue();
        }
    }
    for (int r2 = 0; r2 > rowCount; r2++) {
        Number n = data.getValue(r2, column);
        if (n != null) {
            total += n.doubleValue();
        }
    }
    return total;
}
```



Def-Use table	Nodes
DEF 0:= {data, column} USE 0:= { }	0: Values2D data, int column

DEF 1:= { } USE 1:= {data}	1: Paramchecks
DEF 2:= {total, rowCount} USE 2:= {data}	2: double total, int rowCount
DEF 3:= {r} USE 3:= { }	3: int r = 0
DEF 4:= { } USE 4:= {r, rowCount}	4: r < rowCount
DEF 5:= {n} USE 5:= {data, r, column}	5: Number n
DEF 6:= { } USE 6:= {n}	6: if (n != null)
DEF 7:= {total} USE 7:= {total, n}	7: total +=
DEF 8:= {r} USE 8:= {r}	8: r++
DEF 9:= {r2} USE 9:= { }	9: int r2 = 0
DEF 10:= { } USE 10:= {r2, rowCount}	10: r2 > rowCount
DEF 11:= {n} USE 11:= {data, r2, column}	11: Number n
DEF 12:= { } USE 12:= {n}	12. if (n != null)
DEF 13:= {total} USE 13:= {total, n}	13. total +=
DEF 14:= {r2} USE 14:= {r2}	14. r2++
DEF 15:= { } USE 15:= {total}	15. return total

Variable	Defined at node	dcu	dpu
data	0	{1, 2, 5, 11}	{ }
column	0	{5, 11}	{ }
total	2	{7, 13, 15}	{ }
total	7	{7, 13, 15}	{ }
total	13	{13, 15}	{ }
rowCount	2	{ }	{(4, 5), (4, 9), (10, 11), (10, 15)}
r	3	{5, 8}	{(4, 5), (4, 9)}
r	8	{5, 8}	{(4, 5), (4, 9)}
r2	9	{11, 14}	{(10, 11), (10, 15)}
r2	14	{11, 14}	{(10, 11), (10, 15)}
n	5	{7}	{(6, 7), (6, 8)}
n	11	{13}	{(12, 13), (12, 14)}
Total: 24 dcu, 16 pcu			

```java
// Initializing sample table to use for test cases
@BeforeEach
void setUp() {
    values = mock(Values2D.class);
    when(values.getRowCount()).thenReturn(3);
    when(values.getColumnCount()).thenReturn(2);
    when(values.getValue(0, 0)).thenReturn(3);
    when(values.getValue(0, 1)).thenReturn(4);
    when(values.getValue(1, 0)).thenReturn(7);
    when(values.getValue(1, 1)).thenReturn(8);
    when(values.getValue(2, 0)).thenReturn(5);
    when(values.getValue(2, 1)).thenReturn(1);
}

@Test
// Taken from A2
void ValidColumnValues_SumValues() {
    double result = DataUtilities.calculateColumnTotal(values, 0);
    double expected = 15;

    assertEquals(expected, result);
}
```

```java
@Test
// Taken from A2
void OneValueIsInvalid_ReturnZero() {
    values = mock(Values2D.class);
    when(values.getRowCount()).thenReturn(2);
    when(values.getColumnCount()).thenReturn(2);
    when(values.getValue(0, 0)).thenReturn(3);
    when(values.getValue(0, 1)).thenReturn(4);
    when(values.getValue(1, 0)).thenReturn(null);
    when(values.getValue(1, 1)).thenReturn(8);

    double result = DataUtilities.calculateColumnTotal(values, 0);
    assertEquals(0, result);
}
```

- The first test covers c-uses at nodes (1, 2, 5, 7, 8, 13) and p-uses from nodes {(4, 5), (4, 9), (10, 15), (6, 7)}
- The second test will help cover the missing p-uses from nodes {(6, 8)}
- All unfeasible nodes are excluded

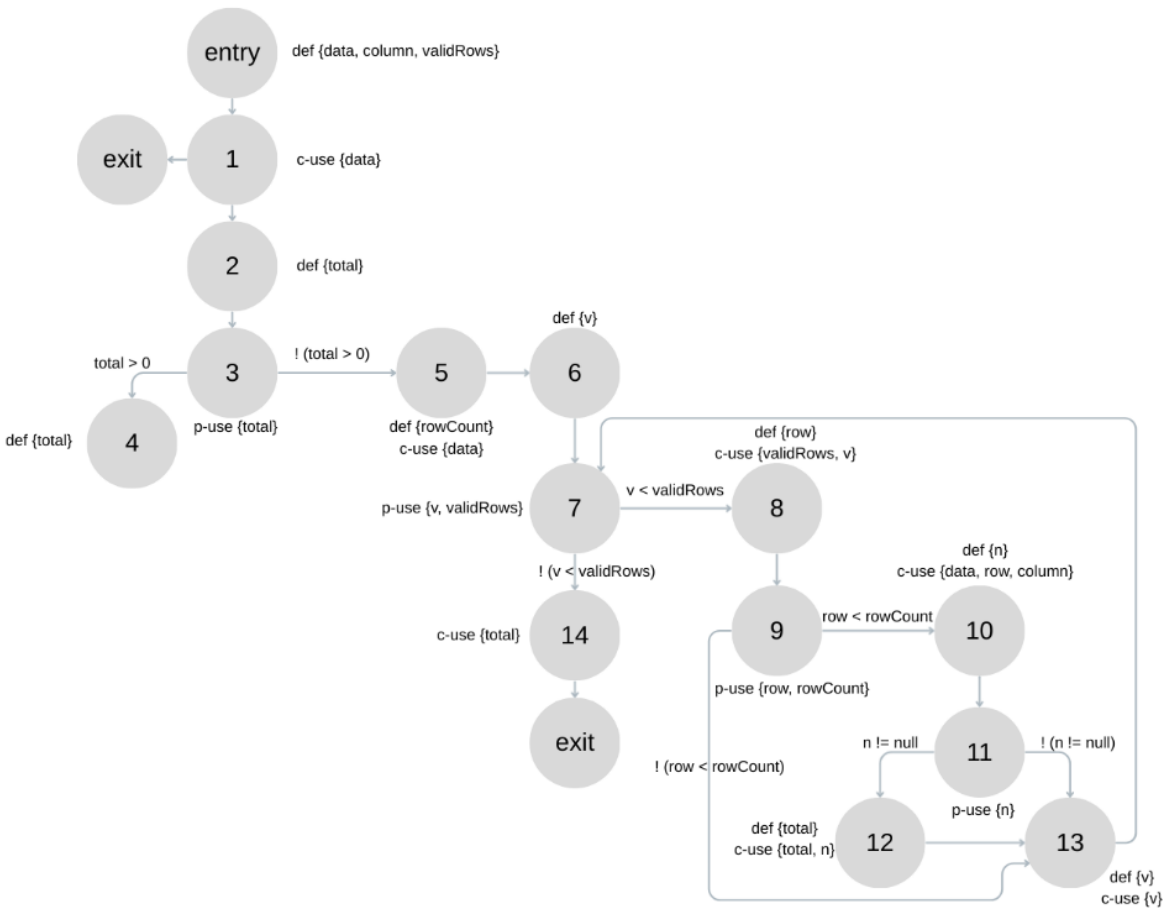
All-uses coverage = $\frac{(13+11)}{((24+16)-(8+5))} \times 100 = 88.89\%$

---

calculateColumnTotal (Values2D data, int column, int[] validRows) method

```java
public static double calculateColumnTotal(Values2D data, int column,
         int[] validRows) {
    ParamChecks.nullNotPermitted(data, "data");
    double total = 0.0;
    if (total > 0){
        total = 100;
    }

    int rowCount = data.getRowCount();
    for (int v = 0; v < validRows.length; v++) {
        int row = validRows[v];
        if (row < rowCount) {
            Number n = data.getValue(row, column);
            if (n != null) {
                total += n.doubleValue();
            }
        }
    }
    return total;
}
```



Def-Use table	Nodes
DEF 0:= {data, column, validRows}	0: Values2D data, int column, int[] validRows
DEF 1:= { } USE 1:= {data}	1: Paramchecks
DEF 2:= {total} USE 2:= { }	2: double total

DEF 3:= { } USE 3:= {total}	3: if (total > 0)
DEF 4:= {total} USE 4:= { }	4: total = 100
DEF 5:= {rowCount} USE 5:= {data}	5: int rowCount
DEF 6:= {v} USE 6:= { }	6: int v = 0
DEF 7:= { } USE 7:= {v, validRows}	7: v < validRows
DEF 8:= {row} USE 8:= {validRows, v}	8: int row
DEF 9:= { } USE 9:= {row, rowCount}	9: if (row < rowCount)
DEF 10:= {n} USE 10:= {data, row, column}	10: Number n
DEF 11:= { } USE 11:= {n}	11: if (n != null)
DEF 12:= {total} USE 12:= {total, n}	12: total +=
DEF 13:= {v} USE 13:= {v}	13: v++
DEF 14:= { } USE 14:= {total}	14: return total

Variable	Defined at node	dcu	dpu
total	2	{12, 14}	{(3, 4), (3, 5)}
total	4	{12, 14}	{(3, 5)}
total	12	{12, 14}	{ }
rowCount	5	{ }	{(9, 10), (9, 13)}
v	6	{7, 12}	{ }
v	13	{12}	{ }
row	8	{9}	{8}
n	10	{11}	{(11, 12), (11, 13)}
Total: 11 dcu, 4 pcu			

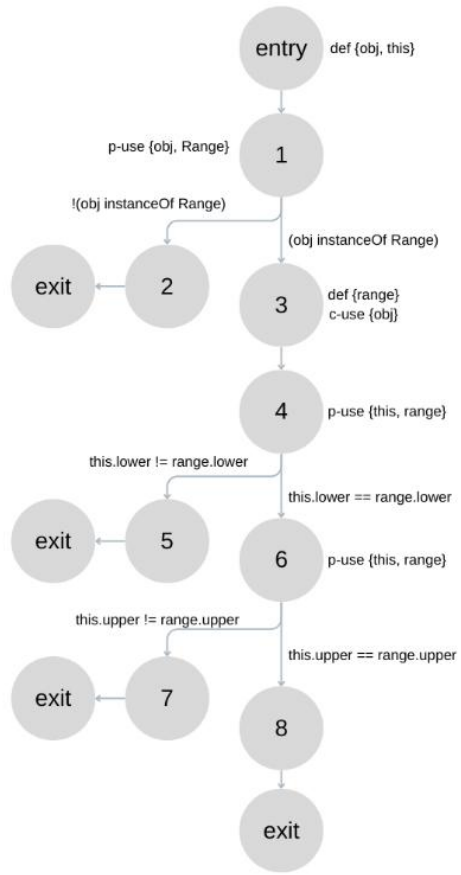
- No tests from assignment 2 cover this method

All-uses coverage = $\frac{(0+0)}{((11+4)-(0+3))} \times 100 = 0\%$

---

equals(Object obj) method

```
@Override
public boolean equals(Object obj) {
    if (!(obj instanceof Range)) {
        return false;
    }
    Range range = (Range) obj;
    if (!(this.lower == range.lower)) {
        return false;
    }
    if (!(this.upper == range.upper)) {
        return false;
    }
    return true;
}
```



Def-Use table	Nodes
DEF 0:= {obj, this} USE 0:= { }	0: Object obj, this
DEF 1:= {} USE 1:= {obj, Range}	1: if (!(obj instanceof Range))
DEF 2:= { } USE 2:= { }	2: return false
DEF 3:= {range} USE 3:= {obj}	3: Range range
DEF 4:= { } USE 4:= {this, range}	4: if (!(this.lower == range.lower))
DEF 5:= { } USE 5:= { }	5: return false
DEF 6:= { } USE 6:= {this, range}	6: if (!(this.upper == range.upper))
DEF 7:= { } USE 7:= { }	7: return false
DEF 8:= { } USE 8:= { }	8: return true

Variable	Defined at node	dcu	dpu
obj	0	{3}	{(1, 2), (1, 3)}
this	0	{ }	{(4, 5), (4, 6), (6, 7), (6, 8)}
range	3	{ }	{(4, 5), (4, 6), (6, 7), (6, 8)}
Total: 1 dcu, 10 pcu			

```java
@Test
void equals_SameObject_ShouldReturnTrue() {
    Range r = new Range(0.0, 1.0);
    assertTrue(r.equals(r));
}

@Test
void equals_DifferentUpperSameLower_ShouldReturnFalse() {
    //  a bug exists because only lower is compared
    Range r1 = new Range(0.0, 1.0);
    Range r2 = new Range(0.0, 999.0);
    assertFalse(r1.equals(r2));
}

@Test
void equals_DifferentLowerSameUpper_ShouldReturnFalse() {
    Range r1 = new Range(0.0, 1.0);
    Range r2 = new Range(-1.0, 1.0);
    assertFalse(r1.equals(r2));
}

@Test
void equals_DifferentType_ShouldReturnFalse() {
    Range r = new Range(0.0, 1.0);
    assertFalse(r.equals("not a range"));
}
```

- The first test covers c-uses at nodes (3) and p-uses from nodes {(1, 3), (4, 6), (6, 8)}
- The second test covers p-uses from nodes {(6, 7)}
- The third test will help cover the missing p-uses from nodes {(4, 5)}
- The last test will cover p-uses from nodes {(1, 2)}

All-uses coverage = $\frac{(1+10)}{((1+10)-(0+0))} \times 100 = 100\%$